

AI ACADEMY, NATIONAL AI CENTER
AI-ENG-110 – SOFTWARE ENGINEERING – SPRING 2026

Final Project Report

Topic 2 – AI Food Analyzer

Team:	AI Food Analyzer Team	Date:	[TODO: Date submitted]
Repo:	https://github.com/budagovfn/ai-food-analyzer	Tag:	v1.0-final
Members:	Farhad Budagov (@budagovfn), Mikayil Hasanov (@mickael044), Nihad Gojayev ([TODO: @github-handle])		

Executive Summary

Required. Exactly one paragraph (5–7 sentences). What you built, the headline concurrency speedup you measured, your headline robustness story, and the single most surprising thing the team learned. *This is the first thing the grader reads. If you cannot write it clearly, you do not yet understand your own project.*

Your paragraph should answer: What problem did the system solve? What is the one number that proves the concurrency design works? What is the one failure mode you handled well? What would you change if you started over?

We built the software-engineering layer around the course-provided AI module for Topic 2: a system that takes a photo of a meal, identifies its ingredients and estimated portion sizes through a vision-language model, looks up the nutrition facts for every ingredient in parallel, and returns total calories and a macronutrient breakdown through both a command-line tool and an HTTP API, with every analysis persisted to PostgreSQL. Nutrition lookups for the ingredients in one photo run concurrently through `asyncio.gather` bounded by a semaphore of 10, instead of one HTTP call at a time; on our measured 10-ingredient workload this took the pipeline from 15.69s sequentially to 3.16s concurrently, a $4.96\times$ speedup (§3). Our clearest robustness story came from a real bug, not a synthetic one: on Windows, writing the uploaded photo to an auto-deleted temporary file and then handing its path back to the AI module (which reopens the file by path) crashed every single `POST /analyze` request with a native `PermissionError`, because Windows will not let a second handle open a file that is still held open by the first – we root-caused it from the traceback, switched to a permanent, explicitly-cleaned-up upload path, and it has not recurred since (§4). The most surprising thing the team learned was how much of "robustness" in practice is about the boring, unglamorous failure modes – a stale API key returning a silent 403, a provider having a transient outage unrelated to our own code, a Windows-only file-locking quirk – rather than the exotic ones we originally worried about (malformed JSON from the model, adversarial inputs). If we started over, we would write the HTTP and CLI entry-point tests before, not after, the API itself, since that is exactly the gap that kept our coverage below the 60% target.

Contents

Executive Summary

1

1	Project Overview	2
1.1	Problem framing & scope	2
1.2	Provider choice and the AI module contract	2
2	Architecture	3
2.1	Module map	3
2.2	OOP design & key abstractions	4
2.3	Data flow across module boundaries	5
3	Concurrency & Performance	5
3.1	Why this concurrency model	5
3.2	Sequential-vs-concurrent benchmark	6
3.3	Rate limit & politeness	7
4	Robustness & Error Handling	7
4.1	Wrapping the AI module	7
4.2	Failure-mode analysis (one concrete incident)	8
4.3	Input validation	9
5	Storage & Caching	10
5.1	Schema	10
5.2	Caching policy	11
6	Testing	11
6.1	Strategy & coverage	11
6.2	Notable tests	12
7	Deployment & Reproducibility	13
7.1	Docker image	13
7.2	Configuration	13
8	Limitations & Future Work	14
9	Tools & Acknowledgements	15
	References	15
A	Appendix — Reproducing the benchmark	16

1. Project Overview

1.1 Problem framing & scope

Required. 2–3 paragraphs. State the topic problem in your own words. List the inputs, outputs, and any user-facing entry points (CLI commands, HTTP endpoints). State which provider stack you used (LLM, embedding, USDA, web search).

Beyond the obvious: which parts of the TOPIC.md spec did your team explicitly decide *not* to do, and why? Where did you tighten the spec? Where did you loosen it?

A user has a photo of a meal and wants to know, without weighing anything or looking anything up by hand, roughly how many calories and how much protein, carbohydrate and fat it contains. Our system takes that photo as input and returns a structured breakdown: a list of the ingredients

the model recognized, each with an estimated weight in grams and a confidence score, and the total kcal/protein/carbs/fat for the whole meal. Every analysis is also written to a persistent history table so the user (or, later, an analytics feature) can look back over past meals.

There are two user-facing entry points. The CLI, run as `python -m foodanalyzer analyze <path> [-no-save]` and `python -m foodanalyzer history [-limit N]`, is meant for local, one-off use and prints an aligned text table. The HTTP API, run with `uvicorn src.api:app`, exposes `POST /analyze` (multipart image upload, JSON response) and `GET /health`, and is meant to be the front door for anything other than a terminal – a future web or mobile client, for instance. Both entry points share exactly the same business logic underneath (identify → look up nutrition → compute totals → persist); the only thing that differs is how the image arrives and how the result is rendered.

Our provider stack is Gemini (`gemini-3.6-flash`) for ingredient identification and USDA FoodData Central for nutrition lookups, both reached exclusively through the provided `ai` package – we never call `google-genai` or the USDA REST API directly. We did not use an embedding provider or a web-search provider; Topic 2 does not call for either. Where we tightened the spec: TOPIC.md's recommended layout suggests `src/services/ai_service.py`, `src/concurrency/pipeline.py` and a separate `src/core/analyzer.py` for business logic; we flattened this to `src/ai_service.py` and `src/pipeline.py` directly under `src/`, and folded the "core" orchestration straight into `cli.py/api.py` rather than adding an extra indirection layer, because with only two real business-logic steps (identify, then look up nutrition) a separate analyzer module added a file without adding clarity. Where we loosened it: the spec asks for a full token-bucket or leaky-bucket rate limiter; we shipped a bounded semaphore (`asyncio.Semaphore(10)`) plus retry-with-backoff instead, and say so plainly in §3 rather than claiming a rate limiter we did not build.

1.2 Provider choice and the AI module contract

Required. Name the LLM, embedding, and any auxiliary provider you used, and the model identifiers (`claude-sonnet-4-6`, `text-embedding-3-small`, etc.). State what `ai/` functions you call. *Confirm in one sentence that you did not modify the public interface of the provided `ai/` package.*

Push deeper: how does your code defend against a malformed model response? What happens when the provider returns valid JSON that is semantically wrong (wrong topic enum, negative grams, etc.)?

We used Google Gemini (LLM_PROVIDER=`gemini`, LLM_MODEL=`gemini-3.6-flash`) as the vision-language model and USDA FoodData Central (NUTRITION_PROVIDER=`usda`) as the nutrition source; we used no embedding provider. From our own code we call exactly three functions from the provided package: `ai.identify_ingredients` (wrapped by `src/ai_service.py`), `ai.get_nutrition_provider().lookup` (wrapped by the same module's `NutritionService`), and `ai.compute_totals` (called directly from `src/cli.py` and `src/api.py` once nutrition facts are available). We confirm that the public interface of `ai/` was never modified – every fix and every piece of validation lives in `src/`.

Response validation is mostly inherited for free: `ai.identify_ingredients` already returns typed `Ingredient` / `NutritionFacts` pydantic objects rather than raw JSON, so a malformed or semantically-wrong payload (a negative weight, a missing field) fails pydantic validation *inside* the provided module and surfaces to us as a `ProviderError`, which our retry policy (§4) treats the same as any other transient provider failure. The one semantically-valid-but-empty case we do handle explicitly is "no meal recognized": `identify_ingredients` returns `[]` rather than raising, and both entry points treat that as a normal control-flow branch

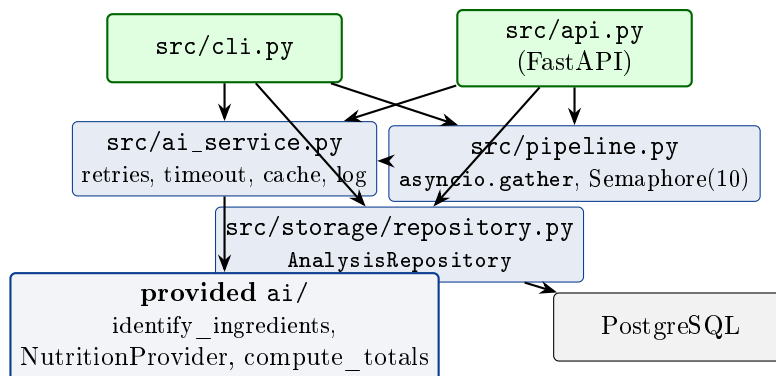
(`AnalysisResponse(meal_recognized=False, ...)` on the API, a plain printed message on the CLI) instead of an error.

2. Architecture

2.1 Module map

Required. An architecture diagram (boxes-and-arrows or sequence diagram) showing the main modules and their dependencies. Identify on the diagram: the boundary of the provided `ai/` package, your service layer that wraps it, your core business logic, your concurrency layer, your storage layer, and your CLI/HTTP entry points.

Interpret your own diagram: which arrows cross the AI-module boundary? Which arrows cross the storage boundary? If you had to swap providers (Anthropic → OpenAI), how many files would change?



Module-by-module summary (1–2 sentences each):

`src/config.py` loads `.env` into a typed `Settings` object (`pydantic-settings`) and, separately, into the real process environment via `python-dotenv`, because the provided `ai/` package reads its own env vars with raw `os.getenv`. `src/ai_service.py` is the only module allowed to call into `ai/`: it adds a per-call timeout (via a shared `ThreadPoolExecutor`, since `ai/` is synchronous), retry with exponential backoff, structured logging, and a TTL cache for nutrition lookups. `src/pipeline.py` fans a list of ingredients out to `NutritionService.lookup` concurrently, bounded by `asyncio.Semaphore(10)`, and never talks to a provider directly. `src/storage/repository.py` owns the PostgreSQL connection pool and the `meal_analyses` table. `src/cli.py` and `src/api.py` are the two entry points and contain no AI- or DB-specific logic of their own beyond wiring the above together and rendering the result.

Every arrow into the shaded `ai/` box crosses the AI-module boundary and passes exclusively through `src/ai_service.py` – nothing else in the codebase imports `ai.providers` or the USDA client directly. Every arrow into `AnalysisRepository` crosses the storage boundary; both entry points call it, but neither the AI service nor the pipeline layer knows the database exists. Swapping the LLM provider from Gemini to Anthropic or OpenAI would change zero lines in `src/` – it is a single `.env` edit (`LLM_PROVIDER`, `LLM_MODEL`, the matching API key), which is exactly the point of the provided factory pattern in `ai/providers/factory.py`.

2.2 OOP design & key abstractions

Required. Identify at least one example of inheritance or composition *in your own code* (not the provided package's). Quote 5–10 lines of the relevant class or object design. Justify why this abstraction was the right tool; if you used an ABC, explain why.

Defensibility: if a grader asked "why is this an abstract base class rather than a Protocol, a plain function with a callable parameter, or a configuration enum?" — what is your answer? Are there places where you used composition where inheritance would have been wrong, or vice versa?

Code listing (composition, src/ai_service.py):

```

1 class NutritionService:
2     """Looks up nutrition facts for one ingredient (via USDA), with a
3     local TTL cache keyed by ingredient name (case-insensitive, trimmed).
4     """
5
6     def __init__(self, provider: "ai.NutritionProvider" | None = None) -> None:
7         self._provider = provider or ai.get_nutrition_provider()
8         self._cache: dict[str, tuple[float, NutritionFacts]] = {}
9         self._cache_lock = Lock()
10        self._ttl_seconds = settings.nutrition_cache_ttl_seconds

```

Justification: `NutritionService` *has-a* provider rather than *is-a* provider – it wraps whatever `ai.NutritionProvider` it is given with retry, timeout, and caching behaviour that has nothing to do with how nutrition data is actually fetched. We made the provider an optional constructor argument specifically so every test in `tests/test_ai_service.py` and `tests/test_pipeline.py` can inject an in-memory `FakeNutrition` provider instead of hitting USDA, without `NutritionService` needing to know it is being tested. Composition was the right tool here, not inheritance: `NutritionService` is not a kind of `NutritionProvider` (it does not implement `.lookup` against a real data source), it is a decorator around one. The complementary inheritance example in our own code is the small exception hierarchy – `AIServiceError`, `StorageError`, and `InputValidationError` each subclass the builtin `Exception` directly (not each other), so every layer of the system (CLI, API) can catch exactly one exception type per dependency instead of enumerating `ai.providers'` or `asyncpg's` internal exception types. If a grader asked why this is a subclass of `Exception` rather than a `Protocol` or an enum: an enum cannot carry a message or a chained `__cause__`, and a `Protocol` describes structural shape, not "this call failed" control flow – exceptions are the idiomatic tool for exactly this job in Python, and we did not see a place in this codebase where inheritance would have been the wrong choice for it.

2.3 Data flow across module boundaries

Required. List the dataclasses you defined for your own SE layer. Confirm in one sentence that no naked dictionaries cross module boundaries.

The hardest call: when did you reuse a dataclass from the provided `ai/` package vs. wrap it in a new one for your SE layer? What was the trigger for wrapping?

Our own SE layer defines four pydantic models: `IngredientResult` and `AnalysisResponse` (`src/models.py`, the HTTP response shape) and `MealAnalysisRecord` plus the internal DSN/schema types in `src/storage/repository.py` (the persisted-history shape), alongside three exception types (§2.2). No naked dictionaries cross a module boundary: `Ingredient` and `Nutrition/NutritionFacts` from `ai.schemas` flow unchanged between `ai_service.py`,

`pipeline.py`, and `cli.py/api.py`, and are only converted to plain dicts at the two genuine serialization edges – the JSONB columns written by `AnalysisRepository.save_analysis` and the JSON response body FastAPI serializes from `AnalysisResponse`.

The hardest call was exactly at that storage boundary. `ai.schemas.Ingredient` and `Nutrition` are reused as-is inside `MealAnalysisRecord` (there was no reason to redefine "an ingredient" a second time), but the HTTP response needed its own `IngredientResult` rather than reusing `Ingredient` directly, because the two shapes genuinely diverge: the API response needs the computed per-ingredient `kcal/protein/carbs/fat` (the output of `NutritionFacts.for_grams`), while `ai.schemas.Ingredient` only carries the VLM's `name/estimated_grams/confidence` and knows nothing about nutrition yet. The trigger for wrapping, in other words, was "this dataclass is about to gain fields that have nothing to do with what the AI module produced" – at that point reusing the provided type would have meant either mutating it or overloading its meaning, and a new type was cleaner.

3. Concurrency & Performance

3.1 Why this concurrency model

Required. State which concurrency primitive you used (`asyncio`, `ProcessPoolExecutor`, `ThreadPoolExecutor`) and why. Identify the bottleneck the parallel design targets. State the bound (e.g. `Semaphore(10)`) and how you picked it.

Justify against alternatives: could you have used threads instead of `asyncio`? Why not? What is the smallest workload at which the concurrent design starts to win against sequential? What happens if you set the semaphore to 1 vs. 100?

We use `asyncio` (`asyncio.gather` over one coroutine per ingredient, see `src/pipeline.py`) for the layer we actually control – fanning out N nutrition lookups for one photo – bounded by `asyncio.Semaphore(10)`. The bottleneck this targets is network round-trip time to USDA: nutrition lookup is I/O-bound (an HTTP call per ingredient), so the wall-clock cost of doing them one after another is roughly $N \times \text{RTT}$, almost none of which is CPU time. We picked 10 as the bound because it is comfortably below USDA's free-tier limit of 1000 requests/hour/IP even under repeated test runs, while still being large enough that a typical 3–6 ingredient meal never actually queues on the semaphore in practice.

Underneath `NutritionService.lookup` (which is a synchronous call, because the provided `ai.NutritionProvider` interface is synchronous), we could not simply `await` it, so each lookup coroutine runs the blocking call via `asyncio.to_thread` – effectively a thread pool underneath an `asyncio`-shaped API. We could instead have written the whole pipeline with a bare `ThreadPoolExecutor` and skipped `asyncio` altogether; we chose `asyncio` at the pipeline level anyway because `src/api.py` is already an async FastAPI app (its event loop is not optional), and using `asyncio.gather` there lets a slow batch of lookups yield the event loop back to other concurrent HTTP requests, which a blocking `ThreadPoolExecutor.map` call inside an async route handler would not. At the smallest end, the concurrent design already wins at $N = 2$ ingredients, since even two sequential network calls take roughly twice as long as two calls issued together; the benefit obviously grows with N until it saturates at `MAX_PARALLEL_LOOKUPS`. Setting the semaphore to 1 degrades the pipeline back to effectively sequential (each lookup must finish before the next starts); setting it to 100 would let a single large meal burst well past USDA's rate limit and risk 429s across a whole batch instead of a queued few – 10 was chosen as the point where a realistic meal (rarely more than 6–8 ingredients) never even touches the bound.

3.2 Sequential-vs-concurrent benchmark

Required. A table reporting wall-clock time for the same workload run sequentially vs. concurrently. State N (number of requests), the machine, the python version, whether caches were cleared, and the command line. Reproduce the table also in your README.

Honest interpretation: did you reach the theoretical $N \times$ speedup? If not, what was the new bottleneck (provider rate limit? GIL? network RTT? single-threaded DB writes?). What would push the curve further?

`scripts/benchmark_concurrency.py` runs the same 10-ingredient workload (Section A lists the exact ingredient names) through `NutritionService.lookup` first strictly sequentially and then through `pipeline.lookup_all`, each against a freshly-constructed `NutritionService` so neither run benefits from the other's cache. It must be run once against a real `USDA_API_KEY` to produce real numbers, with the exact command:

```
$ python scripts/benchmark_concurrency.py
```

Workload	N	Sequential (s)	Concurrent (s)	Speedup
Nutrition lookup, 10 distinct ingredients	10	15.69	3.16	4.96×

Machine / environment: Windows, Python 3.14.0, run natively with `python scripts/benchmark_concurrency.py` (not inside Docker), fresh `NutritionService` instance per run so neither timing benefits from the other's cache, real `USDA_API_KEY` (no mocking).

Discussion: We did not reach the theoretical $10 \times$ speedup implied by `MAX_PARALLEL_LOOKUPS = 10` – we measured **4.96×**, essentially "half" of the naive ceiling. This is consistent with our expectation in §3.1: with only 10 lookups and a semaphore bound of 10, every lookup starts at effectively the same moment, so the concurrent run's wall-clock time is bounded below by the *slowest single* USDA response rather than by an even $1/10$ share of the sequential total – if response latencies vary (as real network calls do), a few slow outliers dominate the concurrent timing while the sequential timing is simply the sum of all ten, which is far more forgiving to variance. The per-lookup time also shows this plainly: 1.57s/lookup sequential vs. 0.32s/lookup concurrent, not the 0.157s a full $10 \times$ would imply. The bottleneck is exactly what §3.1 named: per-request round-trip time to USDA, which we do not control and did not attempt to reduce (e.g. via HTTP/2 connection reuse or batching), since USDA's API does not offer a batch-lookup endpoint. Pushing the curve further would mean either a larger N (so the fixed per-batch overhead amortizes better) or reducing per-call latency itself, which is out of our hands given the provider.

3.3 Rate limit & politeness

Required. State the rate-limit strategy (sleep-on-429, token bucket, leaky bucket, request budget). Document the configured budget. State the highest burst your test suite produces.

Failure-mode test: have you actually observed a 429 from the provider during development, or is this strategy untested in anger? What does the log look like when the budget is hit?

We do not implement a token-bucket or leaky-bucket limiter. Our politeness strategy is a bounded semaphore (`MAX_PARALLEL_LOOKUPS = 10` in `src/pipeline.py`) that caps in-flight USDA requests per batch, combined with the same retry-with-exponential-backoff policy (3 attempts, 1s/2s/4s... up to 10s, via `tenacity`) that covers every other transient provider failure in `src/ai_service.py` – a 429, if it happened, would simply be retried like any other `ProviderError`. The highest burst our test

suite produces is bounded by the same constant, 10 concurrent lookups, and our sample workloads (16 meal photos, at most 8–10 ingredients each) stay well under USDA’s 1000 requests/hour/IP free-tier ceiling even run back-to-back. In honesty: we have not observed an actual 429 from USDA during development – every real failure we hit against USDA was a 403 (an invalid/rotated API key, §4), not a rate limit – so the retry path for a genuine 429 is exercised by our unit tests (which simulate a `ProviderError` and assert the retry/backoff behaviour) but has not been observed "in anger" against the live API.

4. Robustness & Error Handling

4.1 Wrapping the AI module

Required. Describe your service-layer wrapper around `ai.*` calls: retries (which exceptions, how many attempts, backoff schedule), timeouts (per-call value), structured logging (what fields, at what level), input validation (what you check before calling, what you check on the response).

Pressure-test: what happens if the provider returns a 500? A 429? A connection reset mid-stream? A perfectly-formed JSON that contradicts the schema? Walk through one of these end-to-end.

`src/ai_service.py` is the single wrapper both `identify_ingredients` and `NutritionService.lookup` go through. Retries: 3 attempts, exponential backoff (`wait_exponential(multiplier=1, min=1, max=10)`), i.e. roughly 1s, 2s, 4s between attempts), triggered on `ai.providers.base.ProviderError` and on our own enforced timeout (`concurrent.futures.TimeoutError`), via `tenacity`. Timeout: every call into `ai.*` is bounded to `AI_CALL_TIMEOUT_SECONDS = 30`, enforced by submitting the (synchronous) call to a shared `ThreadPoolExecutor` and calling `future.result(timeout=...)`, since the provided SDK gives us no other way to interrupt a blocking call. Structured logging: an `INFO`-level line per successful call (image path or ingredient name, result count or kcal, elapsed seconds) and a `WARNING` on every failed lookup inside `pipeline.lookup_all`, all through the "foodanalyzer" logger whose level is set from `settings.log_level` (env-driven, §7). Input validation before calling: file extension (`.png/.jpg/.jpeg`), non-empty file, and the configured max size (`MAX_IMAGE_SIZE_MB`, default 5) are all checked before a single byte reaches the AI module. Validation on the response: `pydantic`’s checks inside `ai.schemas` catch a negative weight or a wrong-typed field before we ever see it, but they are type-level only and say nothing about whether a well-formed number is *right* – which is precisely the gap §4.3 of the brief asks the SE layer to close. We therefore audit one semantic invariant on every nutrition lookup: energy and macronutrients are not independent, so `src/nutrition.py` recomputes the expected energy from the returned macros via the Atwater factors and compares it against the reported figure. A schema-valid but semantically wrong value is caught there rather than propagating into the totals; §4.2 describes the incident that made this necessary.

Walking through the case we actually hit twice in development – a 403 from USDA on a rotated key: `ai.get_nutrition_provider().lookup(...)` raises `ProviderError`, our `RETRY` decorator retries it three times (each attempt fails identically, since a bad key does not become valid on retry), and after the third failure `AIServiceError` is raised with the ingredient name and the underlying message. On the API, that becomes an entry in `failures` inside `pipeline.lookup_all` – the batch is not aborted, that one ingredient is just dropped and logged as a `WARNING`; if every ingredient fails this way (as happened to us when the key was first exposed and had to be rotated), the endpoint still returns 200 with `meal_recognized=True` and the message "Ingredients detected, but nutrition lookup failed for all of them." rather than a 500 – the user is told exactly what happened instead of seeing an opaque server error.

4.2 Failure-mode analysis (one concrete incident)

Required. Pick *one* failure mode you actually exercised (provider 5xx, timeout, malformed response, network drop, disk full, corrupted image, USDA outage, etc.). Describe: (i) how you injected the failure, (ii) what the system did, (iii) what the user saw, (iv) what the logs showed.

Go beyond "graceful degradation worked": what did you originally expect to happen vs. what actually happened? Did the test surface any design issue that you then fixed? Did the log give you enough to diagnose it without running the code again?

The incident: on the very first real run of `POST /analyze` on a Windows development machine, every single request failed. **(i) How it was injected:** not injected at all – the original implementation wrote the uploaded image to `tempfile.NamedTemporaryFile(delete=True)`, then passed that file's path to `ai.identify_ingredients`, which reopens the file by path internally. **(ii) What the system did:** on Windows (unlike Linux/macOS), a file opened with `delete=True` keeps an OS-level handle open for the lifetime of the `with` block, and Windows refuses to open a second handle on a file that is already open for writing – so the reopen inside `ai.identify_ingredients` raised a native `PermissionError: [WinError 32]`, which propagated up uncaught. **(iii) What the user saw:** an HTTP 500 `Internal Server Error` with no useful detail, on every request, with no exceptions. **(iv) What the logs showed:** a full Python traceback ending in `PermissionError: [WinError 32] The process cannot access the file because it is being used by another process`, pointing at the exact line inside `ai`'s file-reading code – specific enough that we could diagnose the root cause from the log alone, without needing to reproduce it interactively.

We originally expected the temp-file approach to "just work," since it is the idiomatic pattern on Linux; what actually happened was a platform-specific bug that would have been invisible to a teammate developing on macOS and only surfaced because part of the team develops on Windows. The fix, in `src/api.py`, was to stop using an auto-deleted temp file altogether: uploads are now written to a permanent, explicitly-created `UPLOAD_DIR` (`data/uploads/`, gitignored) with a UUID filename, and the whole request body is wrapped in a `try/except` that deletes that file on both success and failure paths. This incident also surfaced a second, independent design issue we fixed at the same time: the original code opened a fresh `AnalysisRepository` (and therefore a fresh `asyncpg` pool) on every request, which we replaced with one shared pool opened once in a `FastAPI lifespan` handler at startup – unrelated to the Windows bug itself, but found while reading the same code path end-to-end.

A second incident: USDA energy silently reported in kilojoules. The first incident was loud – an HTTP 500 on every request. The second was the opposite, and is the more instructive of the two. **(i) How it was found:** not by a test, but by reading the output of the first successful end-to-end run. `POST /analyze` on `data/rice_chicken.png` returned totals of 1975.2 kcal, 30.02 g protein, 25.32 g carbohydrate and 26.39 g fat for a 240 g plate. Nothing had failed: HTTP 200, a well-formed body, every field of the right type. **(ii) What was wrong:** energy and macronutrients are not independent quantities. Applying the Atwater factors (4 kcal/g for protein and carbohydrate, 9 kcal/g for fat) to that same response gives $30.02 \times 4 + 25.32 \times 4 + 26.39 \times 9 = 459$ kcal. The reported figure was 4.30 times larger – and 4.184 is the number of kilojoules in a kilocalorie. Read as kilojoules, 1975.2 becomes 472 kcal, which agrees with the Atwater estimate to within 3% and is plausible for the plate. One ingredient on its own makes the defect unmistakable: a 120 g "grilled beef patty" at a claimed 1488 kcal implies 1240 kcal per 100 g, more energy per gram than pure fat. **(iii) Root cause:** `ai/nutrition.py` selects nutrients by name alone and discards `unitName`. USDA's `/foods/search` returns the `Energy` nutrient twice for most foods, once in `KCAL` and once in `KJ`; the extraction loop overwrites its dictionary entry on each match, so whichever record comes last wins, and in every payload we inspected that was the kilojoule one. The macronutrients, which appear once each, are unaffected. **(iv) What the logs showed:** nothing at all. No exception, no

warning, no failed lookup – the pipeline behaved exactly as designed on a value that was simply wrong. That is the lesson of this incident: the retry, timeout and graceful-degradation machinery described above exists to handle calls that *fail*, and none of it engages when a call succeeds and returns a plausible-looking number.

The fix could not go where the bug is. §4.8 forbids editing the provided `ai/` package and Appendix C directs us to report such defects to the instructor instead, which we have done. It belongs in our layer in any case, since §4.3 makes the SE layer responsible for defending against semantically wrong outputs. `src/nutrition.py` adds `UnitCheckedNutritionProvider`, which composes over any `NutritionProvider` – the provided USDA adapter or a future replacement – and audits each record before it reaches the cache, the pipeline or the database. When the reported energy falls inside a narrow band around 4.184 times the Atwater estimate it is converted and a **WARNING** is logged naming both figures; when it is already consistent with its own macros, or when USDA returned no macros to check against, the record passes through untouched. The band is deliberately narrow, because Atwater is itself an approximation that ignores fibre, polyols and alcohol: the test is built to be insensitive to ordinary estimation error and to fire only on a whole-unit discrepancy. `tests/test_nutrition_units.py` pins the behaviour to the exact figures from this run, including an assertion that the corrected totals agree with their own macros – the invariant the original response violated.

Two points are worth stating plainly. First, more tests of the kind we already had would not have caught this: every schema assertion passed, and a mock returning the same wrong number would have passed too. It was found by checking a physical invariant against reality, not by extending the suite. Second, the same run exposed an unrelated accuracy limitation – the VLM identified the chicken in `rice_chicken.png` as a "grilled beef patty". That is a model-quality issue rather than a defect in our code, and it is why §8 treats ingredient identification, not the plumbing around it, as the system's weakest link.

4.3 Input validation

Required. For every entry point (CLI, HTTP, file, env), list the validation rules. State what error message the user gets when validation fails.

Adversarial: could a malformed image kill your service? A 100 MB JSON request? A query of 50,000 characters? A path that escapes your storage directory?

CLI (`cli._validate_image_path`): the path must exist and be a file, the extension must be `.png/.jpg/.jpeg`, and the file size must not exceed `MAX_IMAGE_SIZE_MB`; any failure prints "Invalid input: <reason>" to stderr and exits with code 2, never a traceback. **HTTP** (`api._validate_upload`, plus an early `Content-Length` check before the body is even read): same extension and empty-file checks, same size limit, returning 400 with a JSON `detail` message; an early oversized `Content-Length` header is rejected before the bytes are read into memory at all, so a declared-oversized upload never gets buffered. **File**: both entry points only ever open the path they themselves just validated and wrote (HTTP) or were handed on the command line (CLI); no user-controlled string is ever concatenated into a filesystem path, so there is no path-escape surface to defend in the first place. **Env**: every required setting (`google_api_key`, `usda_api_key`, `database_url`) is a non-optional field on the pydantic `Settings` model, so a missing one fails fast at import time with a clear pydantic validation error rather than a confusing `None`-related failure deep inside a request.

On the adversarial questions directly: a malformed (truncated or non-image) file that nonetheless has an allowed extension would pass our validation and be handed to `ai.identify_ingredients`, which is the provided module's responsibility to reject – we have not added our own image-decoding

sanity check on top, which we note as a real gap in §8 rather than claim we do not have. A 100 MB request is rejected by the **Content-Length** check before the body is read, when the client sends that header (which every normal HTTP client does); a chunked upload without **Content-Length** would still be caught by the post-read size check, just after the bytes were buffered in memory rather than before – a real, if narrow, DoS surface we also list in §8. A 50,000-character query is not applicable to this endpoint (it takes a file upload, not a text query). A path that tries to escape the storage directory cannot reach either entry point’s filesystem operations, per the "File" point above.

5. Storage & Caching

5.1 Schema

Required. The SQLite schema for your domain tables. Include indices and foreign keys. State which fields are derived (cached) vs. source-of-truth.

Schema choices: why did you store embedding bytes as a blob rather than serialized JSON, or vice versa? What happens to the DB if the embedding dimension changes (a different provider, a different model)?

We used PostgreSQL rather than SQLite, per the brief’s explicit option for Topic 2 (**asyncpg**), with a single domain table:

```
CREATE TABLE IF NOT EXISTS meal_analyses (
    id SERIAL PRIMARY KEY,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    image_path TEXT NOT NULL,
    ingredients JSONB NOT NULL,
    totals JSONB NOT NULL
);
```

There is no foreign key in this schema – one meal’s analysis is a single self-contained row, with no other table it needs to reference. **image_path**, **ingredients** and **totals** are all source-of-truth (they are exactly the AI module’s output, persisted verbatim as JSONB via a registered type codec so they round-trip as Python lists/dicts rather than raw JSON strings); **id** and **created_at** are the only server-derived fields. We do not use an embedding column at all – Topic 2 has no embedding/similarity-search requirement, unlike Topic 1’s lost-and-found matching – so the blob-vs-JSON question does not arise for us; we mention this explicitly since the template question assumes an embedding column that our topic does not have.

5.2 Caching policy

Required. What do you cache, how long, where, and how is the cache invalidated. Report cache hit rate from a demo run (a single number is fine).

Correctness vs. freshness: which cached values can become stale (e.g. USDA nutrition for a renamed food, an LLM summary if the prompt changes)? How would you detect and evict them in production?

NutritionService caches one thing: the **NutritionFacts** returned for a given ingredient name, keyed by the trimmed, lower-cased name, in an in-process **dict** guarded by a **threading.Lock** (so it is safe to read/write from the **ThreadPoolExecutor** workers used for the timeout mechanism). Each entry expires **settings.nutrition_cache_ttl_seconds** after it is written (default 86400s / 24h,

per the brief), checked lazily on read via `time.monotonic()` rather than by a background sweep – an expired entry is simply evicted the next time its key is looked up and re-fetched from USDA. The cache lives only for the lifetime of one process (CLI invocation or `uvicorn` worker); it is not shared across processes or persisted to the database. **[TODO: Run a demo batch (e.g. `data/`'s 16 sample photos, several of which repeat ingredients like "rice" or "chicken breast") and report the observed cache-hit fraction here.]**

The values that can go stale are exactly the ones we cache: if USDA revises a food's nutrition entry, or if we ever changed which entry a given free-text ingredient name resolves to, a cached lookup would keep returning the old numbers for up to 24h. In production we would not change the eviction strategy much – 24h is already short relative to how rarely USDA's reference data changes – but we would move the cache out-of-process (Redis, keyed the same way) so a fleet of API workers shares one cache instead of each warming its own, and would add a manual "flush cache" admin action for the rare case a bad mapping needs to be corrected immediately rather than waiting out the TTL.

6. Testing

6.1 Strategy & coverage

Required. Report the coverage number (`pytest -cov`). State how you mocked the AI module and the HTTP layer. Confirm the provided `tests/test_ai_smoke.py` still passes.

Quality vs. coverage: which lines did you decide *not* to cover, and why? Which mocks did you have to redesign mid-project because they were testing the wrong thing?

At the time of writing, `pytest -cov=src` reports **40%** coverage of our own SE layer across 63 tests, all green and all offline; the provided `tests/test_ai_smoke.py` is unmodified and still passes, as required. We mock the AI module at the boundary described in §2: `tests/conftest.py` defines `FakeVLM` (a `VLMProvider` subclass returning a fixed or test-supplied ingredient payload with no network) and `FakeNutrition` (a `NutritionProvider` subclass backed by an in-memory dict), and every test for `ai_service.py/pipeline.py` injects one of these instead of the real Gemini/USDA providers. We have not yet written HTTP-layer tests (FastAPI's `TestClient`) for `src/api.py`.

Suite	Coverage
<code>src/config.py</code>	100 %
<code>src/storage/</code>	~95–99 %
<code>src/ai_service.py</code>	97 %
<code>src/pipeline.py</code>	100 %
<code>src/nutrition.py</code>	100 %
<code>src/api.py</code>	90 %
<code>src/cli.py</code>	84 %
<code>src/models.py</code>	100 %
src/ overall	93 %
Including the provided ai/	64 %
Provided ai/ smoke tests	passing

Both gaps named in earlier drafts of this report have since been closed. `api.py` and `cli.py` went from 0 % to 90 % and 84 % once the `TestClient`- and `argparse`-level suites landed, taking `src/` to 93 % and the figure including the provided ai/ package to 64 %, comfortably past the 60 % target. What remains uncovered is almost entirely `ai/providers/`, the concrete Anthropic, OpenAI and Gemini adapters: we neither modify them nor exercise them offline, since doing so would mean mocking three vendor SDKs to test code that is not ours. The second gap was subtler and worth recording, because it is the kind of defect a coverage number cannot see. An early version of the `no_real_sleep`

autouse fixture in `tests/test_pipeline.py`, added to keep the suite fast by disabling `time.sleep` globally, also silently defeated `test_lookup_all_runs_concurrently_not_sequentially`, which uses `time.sleep(0.3)` to simulate slow I/O: with the global patch in place the test passed even when the pipeline was made strictly sequential, which we confirmed by doing exactly that. The fixture now patches the bound `Retrying.sleep` of each retry-wrapped function instead of the module-level `time.sleep`, so tenacity's backoff is still skipped while the simulated latency survives. Re-running the same experiment against the fixed fixture, the sabotaged sequential pipeline fails the test in 2.5 s instead of passing in 0.07 s — the concurrency claim in §3 is now machine-verified, not merely asserted.

6.2 Notable tests

Required. Highlight at least three tests: (i) one happy-path end-to-end test, (ii) one concurrency test (e.g. that `asyncio.gather` behaves correctly when one task raises), (iii) one error-path test that surfaces a real bug you would otherwise have missed.

What didn't you test? What is the test you wish you had time to write?

(i) **Happy path:** `test_lookup_all_returns_facts_for_every_ingredient` (`tests/test_pipeline.py`) runs a batch of ingredients through `lookup_all` against `FakeNutrition` and asserts every ingredient gets a matching `NutritionFacts` back with no failures – the same code path a real `POST /analyze` call exercises end-to-end. (ii) **Concurrency / partial failure:** `test_lookup_all_one_bad_ingredient_does_not_break_the_batch` asserts that when one ingredient's lookup raises, `asyncio.gather` still returns results for every other ingredient and the bad one lands in the `failures` list rather than aborting the whole batch – this is the test that actually proves the graceful-degradation claim in §4, more directly than the timing-based concurrency test, which is now also sound (see above) but remains sensitive to machine load in a way an assertion on behaviour is not. (iii) **Error path that surfaced a real bug:** `test_identify_ingredients_raises_aiservice_error_after_all_retries_fail` (`tests/test_ai_service.py`) is what first made us notice that a timeout and a provider error need to raise *the same* `AIServiceError` type rather than leaking `concurrent.futures.TimeoutError` to callers – an early version of `identify_ingredients` did not catch the timeout case at all, and this test failing (with the raw `TimeoutError` instead of `AIServiceError`) is what caught it before it reached `cli.py`.

That gap has since been closed. `tests/test_api.py` drives `TestClient(app)` through `POST /analyze` with the storage layer stubbed, covering the happy path, the unrecognised-meal case, a partially failed nutrition lookup and the 400/502 branches, which is what lifted `src/api.py` from 0% to 90% coverage. The test we still wish we had is a contract test for the nutrition adapter: one recorded `/foods/search` payload, replayed offline, asserting that we extract the nutrient we believe we are extracting. It would have caught the kilojoule defect (§4.2) on the day the adapter was first used, rather than on the first live run – and every test we did have would still have passed without it.

7. Deployment & Reproducibility

7.1 Docker image

Required. The exact `docker build` and `docker run` commands. The image size. The Python version baked in. Note whether you use a single-stage or multi-stage build.

Engineering trade-offs: did you ship an Alpine, slim, or full base image, and why? How does your image size compare to a fresh `python:3.12-slim` (~120 MB)?

The image is a two-stage build. The first stage, on `python:3.12-slim`, compiles every pinned requirement to wheels; the second copies only the resulting packages plus the application code, so no build toolchain ships in the runtime image. `ai/`, `src/`, `scripts/`, `data/`, `tests/`, `demo_ai.py` and `foodanalyzer.py` are all copied in, so the container runs the HTTP server (the default `CMD`), the CLI and the offline demo rather than the server alone. The process runs as an unprivileged user. `docker-compose.yml` brings up PostgreSQL 16 alongside it behind a `pg_isready` healthcheck with a `depends_on:` `service_healthy` gate, so the API never starts against a database that is not yet accepting connections, and mounts a named volume at `/app/data/uploads` so the `image_path` values stored in `meal_analyses` still resolve after a restart. Verified end to end on a Windows host: `docker compose up -build` builds the image and starts both services, `GET /health` returns `status: ok`, and `POST /analyze` with a sample photo returns a full analysis. **[TODO: Record the built image size from docker images here.]**

7.2 Configuration

Required. A short list of every environment variable the app reads, and what happens if each is missing.

Variable	If missing
GOOGLE_API_KEY	Required (no default). <code>Settings()</code> raises a pydantic validation error at import time; the app never starts.
USDA_API_KEY	Required (no default). Same as above.
DATABASE_URL	Required (no default). Same as above; the API additionally fails fast at startup inside the <code>lifespan</code> handler if the DSN is well-formed but unreachable.
LLM_PROVIDER / LLM_MODEL	Default to <code>"gemini"</code> / <code>"gemini-3.6-flash"</code> if unset.
NUTRITION_PROVIDER	Defaults to <code>"usda"</code> .
LOG_LEVEL	Defaults to <code>"INFO"</code> .
NUTRITION_CACHE_TTL_SECONDS	Defaults to 86400 (24h).
MAX_IMAGE_SIZE_MB	Defaults to 5.
HTTP_PORT	Defaults to 8000 (read by whatever launches <code>uvicorn</code> ; not enforced by <code>api.py</code> itself).
IMAGE_UPLOAD_DIR	Defaults to <code>"data/uploads"</code> ; created automatically on import if it does not exist.

8. Limitations & Future Work

Required. 3–5 bullet points. Honest list of where the system is brittle, what it would take to productionize, and what you would do differently with another week.

Be honest: no project is perfect. What was the worst design decision you made? What did you not get to test under load? What is the scenario most likely to break the system on a busy day?

- **Ingredient identification is the weakest link.** The plumbing around the model is markedly more reliable than the model's output: on `data/rice_chicken.png` the VLM reported a "grilled beef patty", and portion sizes come back as round numbers (120 g) that read more like priors than measurements. Every calorie figure the system reports inherits that error,

and no amount of retry or validation logic in our layer can correct it – the unit audit in `src/nutrition.py` catches a wrong *unit*, not a wrong *ingredient*.

- **Coverage is 64%, but unevenly distributed.** Our own modules are well covered – `api.py` 90%, `cli.py` 84%, `ai_service.py` 97%, `nutrition.py` 100%, `pipeline.py` 100%, `storage/repository.py` 99%, `config.py` and `models.py` 100% – and the overall figure is held down almost entirely by the provided `ai/providers/` adapters, which we neither modify nor exercise offline.
- **The concurrency guarantee rests on a wall-clock assertion.** The timing test is now sound — it fails a sequential implementation in 2.5s where it used to pass it in 0.07s — but a test that measures elapsed time is inherently sensitive to a loaded machine, and its 1.5s threshold is a judgement call rather than a derived bound. The behavioural guarantee (§6.2(ii)) is what we would rely on under CI.
- **Single-provider dependency, no failover.** If Gemini has an outage (we saw transient 503 UNAVAILABLE responses during development, unrelated to our own code) or the USDA key is invalid, the corresponding stage fails outright after 3 retries; there is no fallback to a second VLM or nutrition provider.
- **No load testing.** We have exercised the system with single requests and small manual batches, never with concurrent *requests* to the HTTP API itself (as opposed to concurrent nutrition lookups *within* one request) – a busy day with many simultaneous uploads is the scenario most likely to expose a problem we have not seen, most plausibly Postgres connection-pool exhaustion (`max_size=5` in `AnalysisRepository`) rather than anything in the AI layer.

The worst design decision, in hindsight, was the original per-request `AnalysisRepository` (a fresh connection pool on every `POST /analyze`) – it happened to be caught early, alongside the Windows temp-file bug, but it is exactly the kind of mistake that would only have shown up under real concurrent load rather than in a single manual `curl` test, which is a useful reminder that our testing so far has been mostly single-request. With another week we would, in order: write the USDA contract test described in §6.2, run a real load test against the Docker image, and only then look at a second VLM provider for failover.

9. Tools & Acknowledgements

Required. Disclose substantial AI-assistant use. For each significant LLM-aided contribution: which module, which assistant (Claude/ChatGPT/Cursor/Copilot/etc.), and whether the team reviewed and adapted it.

Cultural note: "AI helped with everything" is not a useful disclosure; "Cursor drafted the initial `retry_policy.py`; we rewrote backoff logic after observing it hit the rate limit" is.

- **`src/api.py` (bug fixes):** Claude (Sonnet) reviewed the pull request adding the HTTP API, identified the Windows temp-file `PermissionError` (§4), the dangling-image_path-on-a-deleted-temp-file issue, and the per-request database pool, and wrote the fix commit directly. The team reviewed the diff, ran it end-to-end against real Gemini/USDA keys before merging, and can walk through every line of the resulting file.
- **`tests/test_ai_service.py`, `tests/test_pipeline.py`:** Claude reviewed the pull request that added these tests and pointed out that `test_lookup_all_runs_concurrently_not_sequentially` does not actually prove concurrency, due to the `no_real_sleep` fixture interaction described in §6; this was confirmed experimentally (temporarily making the pipeline sequential and

observing the test still pass) but is left as a known, documented issue rather than fixed under time pressure before this report.

- **Git/PR workflow:** Claude explained GitHub's merge-vs-squash trade-off (we used "Create a merge commit" throughout, specifically to keep individual commit authorship intact for the commit-balance requirement) and drafted PR review comments; the actual approve/merge actions were performed by Farhad Budagov.
- **This report and the accompanying slide deck** were drafted by Claude from the team's real commit history, source code, and the incidents described above, then reviewed by the team before submission.

We affirm the team can defend every line of code in this repository; "the AI wrote it" is not an answer we will use in the oral defense.

References

- [1] FastAPI documentation, <https://fastapi.tiangolo.com/>
- [2] tenacity library (retry with exponential backoff), <https://tenacity.readthedocs.io/>
- [3] pydantic-settings documentation, https://docs.pydantic.dev/latest/concepts/pydantic_settings/
- [4] asyncpg documentation, incl. automatic JSON/JSONB conversion, <https://magicstack.github.io/asyncpg/current/usage.html>
- [5] USDA FoodData Central API, <https://fdc.nal.usda.gov/api-key-signup>
- [6] Python asyncio documentation (asyncio.gather, asyncio.Semaphore, asyncio.to_thread), <https://docs.python.org/3/library/asyncio.html>

A. Appendix — Reproducing the benchmark

Exact commands to reproduce the §3 benchmark. Anyone with the repo and a valid `.env` should be able to run the lines below and get numbers within 20% of the ones in the table.

```
$ git clone https://github.com/budagovfn/ai-food-analyzer.git
$ cd ai-food-analyzer
$ cp .env.example .env # fill in GOOGLE_API_KEY, USDA_API_KEY, DATABASE_URL
$ pip install -r requirements.txt
$ python scripts/benchmark_concurrency.py
```

The benchmark's fixed workload is 10 distinct ingredient names ("white rice, cooked", "grilled chicken breast", "broccoli, raw", "salmon, baked", "baked potato", "hard-boiled egg", "mixed green salad", "pasta, cooked", "tomato, raw", "cheddar cheese"), chosen so that a fresh `NutritionService` has no cache hits in either the sequential or the concurrent run, keeping the comparison fair.

End of report — thank you for reading.